Техническая информация по бронированию игровых мест в BlackBears Play

В клубах blackbears play используется софт **icafe cloud** для управления компьютерами и аккаунтами игроков.
API icafe - https://dev.icafecloud.com/docs/
Запросы надо отправлять из приложения на сервер vibe.blackbearsplay.ru

В приложении используем вызовы API https://dev.icafecloud.com/docs/ + несколько уникальных вызовов, описанных ниже.

Доступы с балансом для проверки работы приложения
test1:test1
test2:test2
test3:test3

Запросы к vibe.blackbearsplay.ru **2**
- /cafes 2
  - Описание 2
  - Пример успешного ответа 2
- /available-pcs-for-booking 2
  - Описание 2
  - Пример данных 3
  - Пример успешного ответа 3
  - Пример ответа с ошибкой 4
- /struct-rooms-icafe 4
  - Описание 4
  - Пример данных 4
  - Пример успешного ответа 5
  - Пример ответа с ошибкой 5
- /all-prices-icafe 6
  - Описание 6
  - Пример использования 6
  - Структура ответа 6
  - Пример данных 8
  - Пример успешного ответа 8
  - Пример ответа с ошибкой 9
- /booking 9
  - Описание 9
  - Параметры в конфиге для этого запроса 10
  - Пример данных 10
  - Ошибка при наслаивании бронирования 10
- /all-books-cafes 11
  - Описание 11
  - Пример данных 11
  - Пример успешного ответа 11
- /icafe-id-for-member 12
  - Описание 12
  - Пример успешного ответа 12

Коды ответов от сервера (Ошибки) **13**

Успешный ответ 13
Коды ошибок - сообщение - комментарий 13
Последовательность вызовов для регистрации игрока 14
Авторизация игрока 14

Запросы к vibe.blackbearsplay.ru

/cafes

Описание

В ответ присылает список клубов для бронирования

Пример успешного ответа

```
{
    "code": 0,
    "message": "Success",
    "data": [
        {
            "address": "Тамбов, Советская 121",
            "icafe_id": 74922
        },
        {
            "address": "Тамбов, Астраханская 2а",
            "icafe_id": 76301
        }
    ]
}
```

/available-pcs-for-booking

Описание

Данный запрос создает массив ПК, проверяет их доступность (Можно ли в дату-время, на которое хотят забронировать - забронировать, и записывается в поле **is_using** *(true - если занят, false - если свободен)*), и ищет ближайшее окно *(и в прошлом, и в будущем, от даты-время на которую хотят забронировать)*.

Ближайшее окно записывается в 4 параметра:
start_date*(Начало даты окна)*,
start_time *(Начало времени окна)*,

end_date *(Конец даты окна)*,
end_time *(Конец времени окна)*

Необязательное поля
isFindWindow - ищет окно, если задан и стоит true
priceName - имя пакета, если выбран пакет, нужно отправлять, так как происходит доп проверка, что в свободном окне пакет работает

Пример данных

Пример запроса

```
/available-pcs-for-booking?
cafeId=74922&
dateStart=2025-12-10&
timeStart=13:00&
mins=60&
isFindWindow=true&
priceName=3 часа/пакет
```

GET параметры запроса

cafeId	74922	ID icafe
dateStart	2025-12-10	Дата, на которую проверяется доступность ПК
timeStart	13:00	Время, на которое проверяется доступность ПК
mins	60	Длительность бронирования
isFindWindow	true	Не обязательное поле, включить выключить поиск окна, если не задан, то окно не ищется
priceName	3 часа/пакет	*(он же product_name)* Не обязательное поле, если выбран пакет, отправляется его общее название для всех зон(в которых он может быть), и происходит доп проверка на то, что пакет может быть забронирован в найденное окно. Если пакет не найден, то свободного окна не будет.

Пример успешного ответа

```
{
    "code": 0,
    "message": "Success",
    "data": {
        "time_frame": "30",
        "pc_list": [
```

```
            {
                "pc_name": "pc01",
                "pc_area_name": "BootCamp 1",
                "pc_box_left": 0,
                "pc_box_top": 0,
                "pc_box_position": "absolute",
                "pc_comment": "",
                "pc_console_type": 0,
                "pc_enabled": 1,
                "pc_group_name": "BootCamp",
                "pc_icafe_id": 74922,
                "price_name": "BootCamap почасовой",
                "is_using": false,
                "start_date": null,
                "start_time": null,
                "end_date": null,
                "end_time": null
            },
            # ... (другие ПК)
        ]
    }
}
```

Пример ответа с ошибкой

```
{
    "code": 419,
    "message": "Empty mins"
}
```

/struct-rooms-icafe

Описание

В ответ отдает данные с icafe + свои поля.
На данный момент добавляется три поля:
pcs_list
color_border
color_text

Пример данных

Пример запроса

```
/struct-rooms-icafe?
cafeId=74922
```

GET параметры

cafeId	74922	ID icafe

Пример успешного ответа

```
{
    "code": 0,
    "message": "Success",
    "data": {
        "rooms": [
            {
                "area_icafe_id": 74922,
                "area_name": "BootCamp 1",
                "area_index": 0,
                "area_frame_x": 10,
                "area_frame_y": 0,
                "area_frame_width": 100,
                "area_frame_height": 360,
                "area_allow_booking": 1,
                "pcs_list": [
                    {
                        "pc_name": "pc01",
                        "pc_area_name": "BootCamp 1",
                        "pc_box_left": 0,
                        "pc_box_top": 0,
                        "pc_box_position": "absolute",
                        "pc_comment": "",
                        "pc_console_type": 0,
                        "pc_enabled": 1,
                        "pc_group_name": "BootCamp",
                        "pc_icafe_id": 74922
                    },
                    # ... (другие ПК)
                ],
                "color_border": "#FFFF00BD",
                "color_text": "#FFFF00BD"
            },
            # ... (другие комнаты)
        ]
    }
}
```

Пример ответа с ошибкой

```
{
    "code": 416,
    "message": "Empty cafeId"
}
```

/all-prices-icafe

Описание

В ответ отдает все родные тарифы, и пакеты (в рамках icafe, это продукты), без лишних полей, и с подсчетом скидки, в зависимости от клиента.

Может в себя принимать не обязательные поля, а именно длительность и дату.

- Если отправить дату, то будет фильтроваться пакеты, которые доступны в данную дату. Например, если 12 марта отправить, то в ответ отправятся пакеты, которые доступны 12 марта, а не все из icafe. Также, если в данный день праздник, и он настроен в конфиге, то будет отдаваться пакет как обычно, только с праздничной *(по факту цена самого дорогого пакета, скорее всего пакета выходных дней, так как берется большая цена)* ценой пакета (а также меняется ID пакета, на ID пакета, у которого взяли самую высокую цену).
- Если отправить длительность, то для родного почасового тарифа, будет считаться итоговая цена на основе длительности.
- Если отправить ID клиента, то будет подсчитываться ценник со скидкой, которая есть у данного клиента.

Пример использования

/all-prices-icafe? cafeId=74922& memberId=312067822505& mins=60& bookingDate=2025-03-09	В ответ придут не все ценники, а только те, которые доступны в данную дату. Также посчитается ценник с учетом скидки, у клиента.
/all-prices-icafe? cafeId=74922& memberId=312067822505& mins=120	В ответ придет все ценники с учетом скидки клиента 312067822505, и длительностью 2 часа, для тарифов, так как для пакетов не влияет длительность, у них своя длительность.
/all-prices-icafe? cafeId=74922& memberId=312067822505&	В ответ придет все ценники с учетом скидки клиента 312067822505, и длительностью 1 час *(так как для тарифа ценник изначально выставлен в 1 час)*.
/all-prices-icafe? cafeId=74922	В ответ придут все ценники, без учета скидки.

Структура ответа

prices		Массив родных почасовых тарифов
price_id	int	ID родного тарифа
price_name	string	Имя тарифа
price_price1	string (“00.00”)	Цена часа тарифа

total_price	string (“00.00”)	Цена с учетом длительности *(если отправлялась в запросе)* и скидки клиента *(если отправлялся ID клиента)*.
group_name	string	К какой группе *(при обсуждении в чате, будет использоваться для определения в какой комнате/зоне работает тариф)* относится тариф
products		Массив пакетов-продуктов
product_id	int	ID пакета
product_name	string	Имя пакета
product_price	string (“00.0000000000000”)	Цена без скидки
product_show_time	string (“00:00-24:00”)	Время работы пакета внутри дня
product_show_weekday	string (“7|1|2|3|4|5|6”)	Работа пакета в дни недели *(для стороны приложения не обязательны, так как со стороны сервера и так проверка)*
is_calc_duration	bool	Параметр, который определяет с каким типом длительности пакет, true - с динамической (то есть должна подсчитываться от времени начала бронирования), false - с фиксированной длительностью. Узнается путем разницы длительностей, и если больше 0, значит длительность динамическая.
duration	string	Длительность пакета в минутах. Длительность на данный момент хранится в названии пакета, после “<<<”, для приложения они вырезаются, и там нормальные названия
duration_min	string	Минимальная длительность пакета, нужна для пакетов с динамической длительностью, если пакет с фиксированной длительностью, то этот параметр будет равен duration
product_show_time_start	string (“00:00”)	Для удобства прилке, начало времени показа пакета
product_show_time_end	string (“23:59”)	Конец показа пакета

total_price	string (“00.00”)	Цена с учетом скидки клиента *(Если отправлялся ID клиента)*.
group_name	string	К какой группе *(при обсуждении в чате, будет использоваться для определения в какой комнате/зоне работает пакет)* относится пакет

Пример данных

Пример запроса

```
/all-prices-icafe?
cafeId=74922&
memberId=312067822505&
bookingDate=2025-03-09&
mins=60
```

GET параметры запроса

cafeId	74922	ID icafe
memberId	312067822505	ID клиента в icafe
bookingDate	2025-03-09	Дата, на которую хотят забронировать. Если её отправить, в ответ даст пакеты, которые доступны на данный день. Так же, если в этот день праздник, то будет показываться его праздничная *(по факту выходного пакета)* цена пакета.
mins	60	Не обязательное поле mins. Если оно пустое, то подсчитывание скидки для родного тарифа будет за 60 минут, то есть стандартный ценник. price_price1 не меняется, так как это поле icafe, ценник сам итоговый в total_price, с учетом скидки и времени.

Пример успешного ответа


```
{
    "code": 0,
    "message": "Success",
    "data": {
        "prices": [
            {
                "price_id": 62853,
```

```
                "price_name": "GameZone почасовой",
                "price_price1": "120.00",
                "duration": "60",
                "total_price": "102.00",
                "group_name": "GameZone"
            },
            # ... (другие тарифы)
        ],
        "products": [
            {
                "product_id": 1591240869,
                "product_name": "Booking BootCamp 3 часа",
                "product_price": "1234.0000000000",
                "product_show_time": "00:00-23:59",
                "product_show_weekday": "1|2|3|4",
                "product_enable_client": 0,
                "product_enable_discount": 1,
                "duration": "180",
                "duration_min": "180",
                "is_calc_duration": false,
                "product_show_time_start": "00:00",
                "product_show_time_end": "23:59",
                "total_price": "1234.0000000000",
                "group_name": "BootCamp"
            },
            # ... (другие пакеты)
        ]
    }
}
```


Пример ответа с ошибкой


```
{
    "code": 416,
    "message": "Empty cafeId"
}
```


/booking

Описание

Порядок работы:

1 - Доп проверка наслаивания бронирования (так как были случаи наслаивания), которую можно отключить в конфиге сервера.
2 - Инициализация бронирования
3 - происходит “развилка”, если бронь без пакета, то просто идет отправка запроса на бронирование в icafe (деньги списываются с баланса на стороне icafe при выполнении этого запроса, где в приоритете

списывания с бонус баланса, а потом уже, если не хватает с обычного баланса). Так же проверка что успешно бронь произошла, и в ответ будет либо успех либо нет, на этой ветке с развилки.
4 - на другой ветки, этой развилки, сначала накидывается на бонус баланс деньги (сумма берется из 2 пункта, то есть при инициализации известно сколько будет стоить бронирование) (есть проверка что, успешно отработал запрос, то есть, проверка на то, что деньги закинулись), чтобы бесплатно забронировать (так же есть проверка на успех запроса, если не успех, в ответ отправится что не удалось забронировать), после происходит покупка пакета-продукта, и происходит проверка на то, что получилось купить пакет-продукт, если нет, бронь отменяется, если да, то в ответ отправляется успех и отправляется два запроса “pushClientStatus” (метод в коде, который делает запрос) и “orderStatistic”.
5 - все, запрос закончился.

Параметры в конфиге для этого запроса

is_check_booking_pc	Флаг на включение и отключение доп проверки наслаивания/пересечения бронирования.

Пример данных

Пример запроса

/booking

POST параметры запроса

icafe_id	74922	ID icafe в котором бронировать (по дефолту стоит 74922, так как старые версии приложения не отправляют его)
pc_name	pc24	
member_account	pablo4	
member_id	312077721789	
start_date	2026-01-22	
start_time	17:00	
mins	60	
rand_key	12345678911	
key	cda9bec10f020985742771d2f609fb4d	

Ошибка при наслаивании бронирования

Ответ если найдено наслаивание, или затрагивания времени уже забронированного места, будет выглядеть так. (также учитывается время на доигрование, которое берется из конфига сервера)

```
{
    "code": 600,
    "message": "Booking is occupied"
```

```
}
```

/all-books-cafes

Описание

Собирает все брони, со всех клубов и отправляет в ответ. Брони группируются по id клуба.

Если указано имя клиента, брони будут отфильтрованны по нику. Если брони у клиента нет в каком то из клубов, то пустого массива броней этого клуба не будет.
Пример: если забронировано у клиента только в 74922 клубе, то клубы 76301 и др, с пустыми массивами не будут в ответе.

Пример данных

Пример запроса

```
/all-books-member?
memberAccount=pablo4
```

GET параметры запроса

memberAccount	pablo4	Имя клиента из icafe, с таким же названием параметра. Если его указать, то вернет отфильтрованные под клиента брони. *(Необязателен, без него все брони со всех клубов вернуться)*

Пример успешного ответа

```
{
    "code": 0,
    "message": "Success",
    "data": {
        "74922": [
            {
                "product_id": 76940053,
                "product_pc_name": "pc25",
                "product_available_date_local_from": "2026-03-20 09:00:00",
                "product_available_date_local_to": "2026-03-20 10:00:00",
                "product_mins": 60,
                "product_description": "Почасовая@912753",
                "member_offer_id": 73795193065,
                "member_account": "pablo4"
            },
            // др брони для этого клуба
```

```
        ],
        "76301": [
            {
                "product_id": 76940049,
                "product_pc_name": "PC25",
                "product_available_date_local_from": "2026-03-20 09:00:00",
                "product_available_date_local_to": "2026-03-20 10:00:00",
                "product_mins": 60,
                "product_description": "Почасовая@707501",
                "member_offer_id": 73795193061,
                "member_account": "pablo4"
            },
            // др брони для этого клуба
        ],
       // др клубы
    }
}
```


/icafe-id-for-member

Описание

Возвращает icafe id, который отвечает за запросы по клиентам (создание, получение и тд.).

Пример успешного ответа


```
{
    "code": 0,
    "message": "Success",
    "data": {
        "icafe_id": "74922"
    }
}
```

Коды ответов от сервера (Ошибки)

Данные коды ошибок и их сообщения, реализованы классом **Response**. Эта информация с кодами и сообщениями будет в теле HTTP, если сам HTTP ответ = 200.

Успешный ответ

Код успешного ответа - 0 *(число ноль)*. Также в успешном ответе в следующем поле пишется сообщение “Success”, и передаются данные.

Пример

```
{
    "code": 0,
    "message": "Success",
    "data": {...}
}
```

Коды ошибок - сообщение - комментарий

Код	Сообщение	Комментарий
1	Error	Может присылаться на любом запросе, в любой момент, когда нет записи конкретной ошибки
2	Service Unavailable	Может прийти в любой момент на любом запросе, когда сервер вырубили через конфиг. *(в будущем может быть отдельная ошибка на отключение отдельных запросов, а не всего сервера)*
411	Empty phone	
412	Empty email	
413	Empty client_id	Отсутствует или пустой POST параметр *(внутри HTTP тела/body)*
414	Empty value	
415	Empty currency	
416	Empty cafeId	Отсутствует или пустой GET параметр
417	Empty dateStart	Отсутствует или пустой GET параметр
418	Empty timeStart	Отсутствует или пустой GET параметр
419	Empty mins	

451	Incorrect phone	
452	Incorrect email	
453	Incorrect client_id	
454	Incorrect value	
455	Incorrect currency	
600	Booking is occupied	Может прийти при запросе бронирования, означает что “на это время, этот пк уже забронирован”

Последовательность вызовов для регистрации игрока

Регистрацию упростили до такой последовательности:
1 - POST {{baseURL}}/api/v2/cafe/$cafeId/members
2 - POST {{baseURL}}/request-sms
передаем {member_id: ID }
3 - POST {{baseURL}}/verify
передаем {member_id: ID }

Авторизация игрока

Отправляем запрос на {{baseURL}}/login
Передается username и пароль в открытом виде